Integración de APIs con Agente

Poblado automático de la API del refrigerador inteligente "Mike" a partir de tickets de compra usando MinIO, FastMCP y un agente de IA

Alumno: _____

Materia: _____

Profesor: _____

Fecha: 14/10/2026

1. Introducción

El proyecto **Mike API** es un backend en FastAPI para un refrigerador inteligente: gestiona usuarios, refrigeradores, alimentos, tickets de compra, imágenes, sensores y cámaras, con persistencia en PostgreSQL y archivos en MinIO. Capturar a mano cada producto comprado es lento y propenso a errores, así que esta actividad automatiza el registro: un **agente de IA** lee la foto de un ticket, interpreta su contenido y llena las APIs con los datos y las imágenes correspondientes.

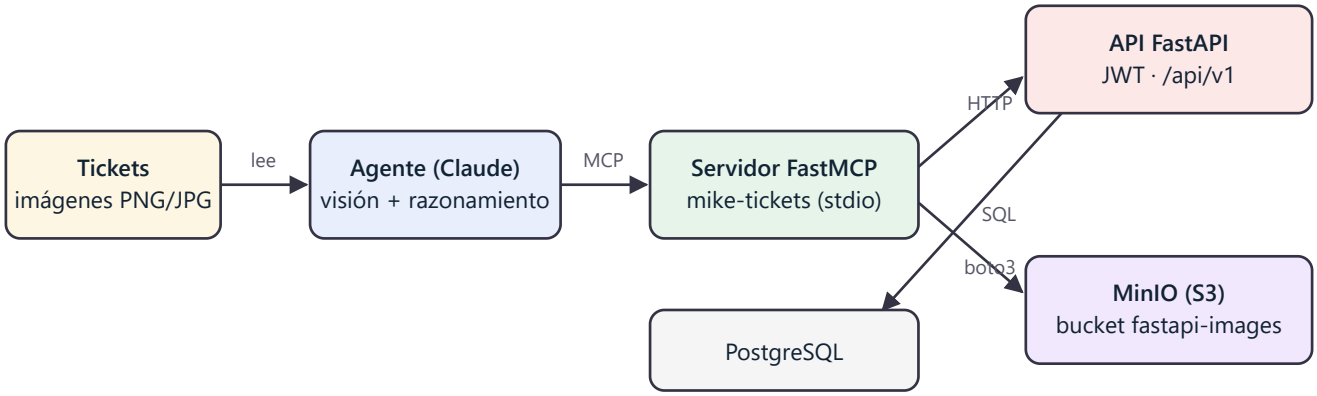
Para que el agente pueda actuar sobre el sistema se construyó un **servidor MCP** (Model Context Protocol) con la librería **FastMCP**. Este servidor expone herramientas bien definidas que encapsulan la autenticación JWT, las llamadas HTTP a la API y la subida de archivos a **MinIO**, un almacenamiento de objetos compatible con Amazon S3.

Objetivos:

- Exponer las APIs de tickets, alimentos e imágenes como herramientas MCP reutilizables por cualquier agente.
- Guardar las imágenes de los tickets y de los productos en MinIO y referenciarlas desde la base de datos.
- Hacer que el agente procese tickets de prueba de principio a fin, sin capturar datos a mano.

2. Desarrollo

2.1 Arquitectura



El agente nunca habla directamente con la base de datos ni con MinIO: todo pasa por las herramientas del servidor MCP. El servidor inicia sesión en la API con un usuario de servicio (`MCP_API_USER` en `.env`), renueva el token si expira y usa `boto3` para subir archivos al bucket `fastapi-images`.

2.2 Infraestructura

- **PostgreSQL 15** y **MinIO** levantados con `docker compose up -d` (puertos 5433, 9000 y 9001).
- **API FastAPI** con `uvicorn main:app --port 8000`; rutas bajo `/api/v1`.
- Entorno virtual de Python 3.14 con `fastmcp 4.1`, `boto3`, `httpx` y `Pillow`.

2.3 Servidor FastMCP (`mcp_server/server.py`)

Herramienta	Qué hace	API / servicio
listar_tickets_pendientes	Lista los tickets en `tickets/` que aún no están registrados y las fotos de productos	GET /purchase-tickets
obtener_o_crear_refrigerador	Busca o crea el refrigerador del usuario	GET/POST /refrigerators
registrar_ticket	Sube la imagen del ticket a MinIO y crea el PurchaseTicket	MinIO + POST /purchase-tickets
registrar_alimento	Crea el Food (calcula fresco, por caducar o caducado), vincula la imagen del ticket y sube la foto del producto	POST /foods, MinIO + POST /food-images
marcar_ticket_procesado	Marca el ticket como procesado	PUT /purchase-tickets/{id}
resumen_refrigerador	Resume los tickets, alimentos e imágenes registrados	GET (varios)

Ejemplo de herramienta: FastMCP genera el esquema de entrada a partir de los *type hints* y el docstring se convierte en la descripción que lee el agente.

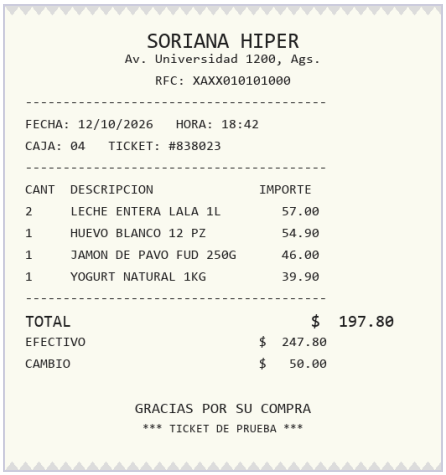
```
@mcp.tool
def registrar_ticket(
    refrigerator_id: int,
    image_path: str,
    store_name: str,
    purchase_date: date,
    raw_text: str,
) -> dict[str, Any]:
    """Sube la imagen del ticket a MinIO y crea el PurchaseTicket en la API.

    image_path: nombre del archivo dentro de tickets/ (ej. 'ticket_soriana.png').
    raw_text: texto transcrito del ticket (productos, precios, total).
    """
    url = _upload(image_path, "tickets")
    return api.request("POST", "/purchase-tickets/", json={
        "user_id": api.user_id,
        "refrigerator_id": refrigerator_id,
        "store_name": store_name,
        "purchase_date": purchase_date.isoformat(),
        "ticket_image_url": url,
        "raw_text": f"archivo: {Path(image_path).name}\n{raw_text}",
        "processed_at": False,
    })
```

El servidor se registra para el agente en `.mcp.json`, así Claude Code lo carga automáticamente al abrir el proyecto.

2.4 Tickets de prueba

Con `tickets/generar_tickets.py` (Pillow) se generaron tres tickets falsos de tiendas distintas y doce fotos ilustrativas de productos.



SORIANA HIPER
Av. Universidad 1200, Ags.
RFC: XAXX010101000

FECHA: 12/10/2026 HORA: 18:42
CAJA: 04 TICKET: #838023

CANT	DESCRIPCION	IMPORTE
2	LECHE ENTERA LALA 1L	57.00
1	HUEVO BLANCO 12 PZ	54.90
1	JAMON DE PAVO FUD 250G	46.00
1	YOGURT NATURAL 1KG	39.90

TOTAL $ 197.80
EFECTIVO $ 247.80
CAMBIO $ 50.00

GRACIAS POR SU COMPRA
*** TICKET DE PRUEBA ***

ticket_soriana.png



WALMART SUPERCENTER
Blvd. Aguascalientes 4512
RFC: XAXX010101000

FECHA: 09/10/2026 HORA: 18:42
CAJA: 04 TICKET: #842390

CANT	DESCRIPCION	IMPORTE
1.2	POLLO PECHUGA KG	154.80
1.0	JITOMATE SALADET KG	32.00
1	LECHUGA ROMANA PZ	22.50
1	QUESO PANELA 400G	68.00
2	REFRESCO COLA 2L	72.00

TOTAL $ 349.30
EFECTIVO $ 399.30
CAMBIO $ 50.00

GRACIAS POR SU COMPRA
*** TICKET DE PRUEBA ***

ticket_walmart.png



OXXO
Calle Madero 301, Centro
RFC: XAXX010101000

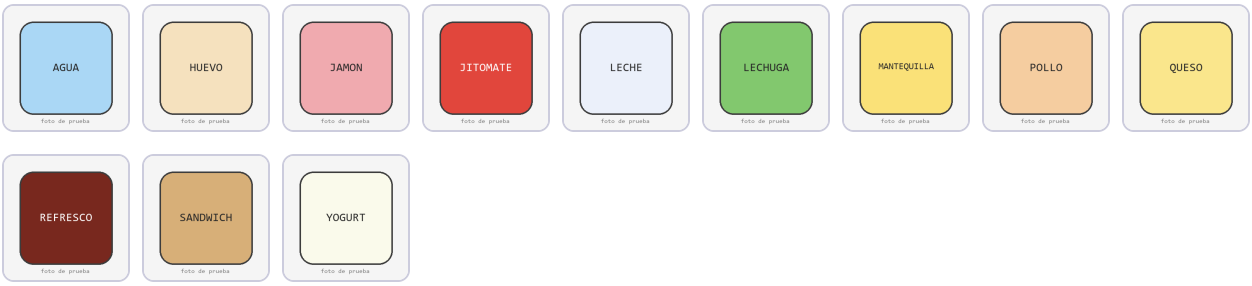
FECHA: 14/10/2026 HORA: 18:42
CAJA: 04 TICKET: #724084

CANT	DESCRIPCION	IMPORTE
2	AGUA NATURAL 1.5L	34.00
1	MANTEQUILLA 90G	24.50
1	SANDWICH JAMON QUESO	42.00

TOTAL $ 100.50
EFECTIVO $ 150.50
CAMBIO $ 50.00

GRACIAS POR SU COMPRA
*** TICKET DE PRUEBA ***

ticket_oxxo.png



2.5 El agente

El agente es **Claude** conectado al servidor MCP. Las instrucciones están en `agente/prompt.md`:

1. Consulta los tickets pendientes y obtiene el refrigerador.
2. Abre cada imagen y extrae, con visión, la tienda, la fecha y los productos con cantidad, unidad e importe.
3. Normaliza los datos: nombre legible, categoría, unidad (kg, L o pz) y la fecha en formato ISO.
4. Como los tickets no traen fecha de caducidad, la estima con la vida útil típica de cada producto.
5. Registra el ticket, cada alimento con sus imágenes, y marca el ticket como procesado.

Se ejecuta con `agente\run_agente.ps1` (Claude Code en modo headless con `--mcp-config .mcp.json`). La sesión ejecutada también quedó como script reproducible en `agente/sesion_agente.py`, y cada llamada a herramienta con su respuesta se registró en `agente/ejecucion_agente.log`. Extracto:

```
>>> listar_tickets_pendientes({})
{
  "carpeta": "C:\\Users\\elihu\\Videos\\lab10\\tickets",
  "pendientes": [
    "ticket_oxxo.png",
    "ticket_soriana.png",
    "ticket_walmart.png"
  ],
  "ya_registrados": [],
  "fotos_de_productos": [
    "productos/agua.png",
    "productos/huevo.png",
    "productos/jamon.png",
    "productos/jitomate.png",
    "productos/leche.png",
    "productos/lechuga.png",
    "productos/mantequilla.png",
    "productos/pollo.png",
    "productos/queso.png",
    "productos/refresco.png",
    "productos/sandwich.png",
    "productos/yogurt.png"
  ]
}

>>> obtener_o_crear_refrigerador({})
{
  "user_id": 1,
  "name": "Refrigerador principal",
  "brand": "Mabe",
  "esp32_device_id": "esp32-8f2a065d",
  "capacity_liters": 400,
  "is_active": true,
  "id": 1,
...
```

2.6 Resultados

1	3	12	24
refrigerador	tickets	alimentos	imágenes (12 de ticket, 12 de producto)

Tickets registrados (la imagen se guardó en MinIO):

ID	Tienda	Fecha	Procesado	Objeto en MinIO
1	Soriana Hiper	2026-10-12	Sí	`fastapi-images/tickets/52725f8bc17142ed9c7fb972d2d4e91d.png`
2	Walmart Supercenter	2026-10-09	Sí	`fastapi-images/tickets/cd4bbc56a50d4e3a9652c9b370a08747.png`
3	OXXO	2026-10-14	Sí	`fastapi-images/tickets/3f0c45b9302645d6aca2b4cc3b4d81ea.png`

Alimentos registrados por el agente:

ID	Alimento	Categoría	Cantidad	Importe	Tienda	Caduca	Estado
1	Leche entera Lala 1L	Lácteos	2 L	$57.00	Soriana Hiper	2026-10-19	fresco
2	Huevo blanco 12 pz	Huevo	12 pz	$54.90	Soriana Hiper	2026-11-02	fresco
3	Jamón de pavo FUD 250g	Carnes frías	0.25 kg	$46.00	Soriana Hiper	2026-10-19	fresco
4	Yogurt natural 1kg	Lácteos	1 kg	$39.90	Soriana Hiper	2026-10-26	fresco
5	Pechuga de pollo	Carnes	1.2 kg	$154.80	Walmart Supercenter	2026-10-11	caducado
6	Jitomate saladet	Frutas y verduras	1 kg	$32.00	Walmart Supercenter	2026-10-16	por caducar
7	Lechuga romana	Frutas y verduras	1 pz	$22.50	Walmart Supercenter	2026-10-14	por caducar
8	Queso panela 400g	Lácteos	0.4 kg	$68.00	Walmart Supercenter	2026-10-19	fresco
9	Refresco de cola 2L	Bebidas	4 L	$72.00	Walmart Supercenter	2027-04-07	fresco
10	Agua natural 1.5L	Bebidas	3 L	$34.00	OXXO	2027-10-14	fresco
11	Mantequilla 90g	Lácteos	0.09 kg	$24.50	OXXO	2026-12-13	fresco
12	Sándwich jamón y queso	Preparados	1 pz	$42.00	OXXO	2026-10-16	por caducar

Estados al 14/10/2026: 8 frescos, 3 por caducar y 1 caducado. La pechuga de pollo, comprada el 09/10, ya superó los 2 días de vida útil en refrigeración.

3. Conclusión

La integración cumplió su objetivo: a partir de tres imágenes de tickets, el agente registró de forma autónoma 3 tickets, 12 alimentos y 24 imágenes de alimentos. Además, guardó 15 archivos en MinIO: los 3 tickets y las 12 fotos de productos. Nadie tuvo que capturar datos a mano: el agente interpretó texto no estructurado, lo normalizó y decidió qué herramienta usar en cada paso.

El servidor MCP fue la pieza clave: separa la lógica de negocio y las credenciales del modelo de IA. El agente solo conoce herramientas con nombres, parámetros tipados y descripciones claras, mientras que la autenticación JWT, las URLs y el acceso a MinIO quedan del lado del servidor. Esto hace al sistema más seguro y fácil de mantener. Además, el mismo servidor puede usarse con otros clientes compatibles con MCP.

Mejoras posibles: leer la fecha de caducidad desde fotos de las etiquetas; agregar un endpoint de subida de imágenes a la API para no acceder a MinIO desde el servidor MCP; usar URLs firmadas para mostrar las imágenes al cliente; y procesar tickets reales automáticamente cuando se suban desde la app o desde la cámara del ESP32.